

计算理论与算法分析

CH1

时间复杂性

假设某算法在输入规模为 n 时计算时间为 $T(n) = 3 * 2^n$ 。在某台计算机上实现并完成该算法的时间为 t 秒。现在另一计算机，其运行速度为第一台的64倍，那么在这台新机器上用同一算法在 t 秒内能解输入规模为多少的问题？

$64 = 2^6, T' = 64 * T(n) = 3 * 2^{n+6}$,所以可以处理规模为 $n+6$ 的问题。

运行时间

存在自然数 n_0 和正常数 c ,使得对所有 $n \geq n_0$,都有...

$O(f(n))$: 上界, $g(n) \leq cf(n)$

$\Omega(f(n))$: 下界, $g(n) \geq cf(n)$

$\Theta(f(n))$: 确界, 存在自然数 n_0 和正常数 c_1, c_2 ,对所有 $n \geq n_0$,都有 $c_1 f(n) \leq g(n) \leq c_2 f(n)$ 。

鸽巢原理

n 只鸽子进入 m 个巢, $n > m$, 则至少一个鸽巢至少有 $\lfloor (n - 1) / m \rfloor + 1$ 个鸽子。

CH2 分治算法

递归函数

总体思想

分——治——合

子问题相互独立, 无重复, 可递归求解

子问题规模大致相等, 合并后能得到原问题的解

时间复杂度

分治法把规模为 n 的

二分法

求最大最小元

递归分成两部分, 各求两部分的最值, 合并时再比较一次两部分的最值

快速排序

选择数组第一个, 中间, 最后一个三个数的中间值作为pivot

递归处理

$O(n \log n)$

归并排序

$O(n \log n)$

循环赛日程表

每个人每天最多和一个人比赛，而且每个人必须和其他人都进行过比赛

设一共有 n 人，当 n 为偶数时需要进行 $n-1$ 天， n 为奇数时需要进行 n 天（此时可以转换为 $n+1$ 人偶数的情况）

分：不断对人数除以二

治：直到分为2人一组，这时每个组各自进行比赛

合：

大整数乘法

把每个整数分成两块，一共四块， $AB*CD$ ，只算三次乘法，然后通过公式转换和一些权重得到最后结果

练习：采用分治算法解决十进制大整数的乘法

已知 $X=2368$ ， $y=3925$ ，求 XY 。

$XY=u+v \times 10^2 + w \times 10^4$ ，求 u,v,w

解：取基 $d=10$ ， $A=23$ ， $B=68$ ， $C=39$ ， $D=25$ ，则

$$\textcircled{1} AC=23 \times 39=897$$

$$\textcircled{2} BD=68 \times 25=1700$$

$$\begin{aligned}\textcircled{3} (A-B)(D-C)+AC+BD \\ &= (23-68)(25-39)+897+1700 \\ &= 45 \times 14 + 897 + 1700 \\ &= 3227\end{aligned}$$

$$\therefore XY=1700+3227 \times 10^2 + 897 \times 10^4 = 9294400$$

线性时间选择

找第 k 小的数：

把数组五个五个分一组，每组排序后取第三个值，再把这些第三个值组成一个数组找这个的中位数，作为整个数组的pivot，根据pivot划分数组，把小的扔到左边，大的扔到右边，递归。

最大字段和

分成左右两段递归，最大字段可能出现在左边、右边、或者中间连接处，对中间的情况需要求左边最大后缀和+右边最大前缀和

一维分治

二维分治

动态规划

动态规划通常按这几个步骤做：

1. **定义状态**：用一个数组或函数表示“某个子问题的最优值”。
2. **找状态转移方程**：大问题如何由小问题得到。
3. **确定边界条件**：最小子问题的答案是什么。
4. **按合适顺序计算**：通常从小规模到大规模。
5. **记录决策，构造最优解**：不仅求最优值，还能恢复路径、方案或序列。

矩阵连乘问题

0-1 背包

最长公共子序列

多段图最短路径

货郎担问题

贪心算法

分数背包

活动安排-多机调度-车间计划排序

最优装载

哈夫曼编码

最小生成树

回溯算法

在DFS基础上添加了判断：发现这条路没有希望就提前撤退

先定义解空间

像0-1背包“从 n 个元素中选子集”的问题叫做 **子集树问题**

TSP 是排列顺序，所以是 **排列树问题**

显隐约束

显约束是每个变量你自己能取什么值

隐约束是不同变量共同要满足的条件

状态空间树

回溯法通常把所有可能解组织成一棵树，叫 **状态空间树**。

回溯法不是一上来生成所有解，而是在搜索过程中动态生成这棵树。

基本步骤

1. 定义解空间
2. 确定解空间结构：判断是子集树，排列树还是其他结构
3. 设计约束和限界函数：减掉不满足条件的或者不可能得到最优解的分支
4. 用深度优先方式搜索

子集树模板

选若干元素使目标最优

```
1 void backtrack(int t) {
2     if (t > n) {
3         // 到达叶子，更新答案
4         return;
5     }
6     // 选择第 t 个元素
7     x[t] = 1;
8     if (constraint(t) && bound(t)) {
9         backtrack(t + 1);
10    }
11    // 不选择第 t 个元素
12    x[t] = 0;
13    if (constraint(t) && bound(t)) {
14        backtrack(t + 1);
15    }
16 }
```

排列树模板

第 t 层确定第 t 个位置放谁

```
1 void backtrack(int t) {
2     if (t > n) {
3         // 得到一个完整排列
4         return;
5     }
```

```

6         for (int i = t; i <= n; i++) {
7             swap(x[t], x[i]);
8
9             if (constraint(t) && bound(t)) {
10                 backtrack(t + 1);
11             }
12             swap(x[t], x[i]); // 还原
13         }
14     }

```

0-1背包

TSP旅行商

n皇后问题

图着色问题

最大团问题

完全子图——团——最大团

空子图——独立集——最大独立集

如果想把顶点 i 加入当前团，必须满足：顶点 i 和当前团中已经选入的所有顶点都有边。

计算理论

ch1

字符串与语言

字母表是任意一个有限集， $\Sigma = \{0, 1\}$ ，其中的元素是**符号**，**字符串**是**符号**组成的有限序列

字符串的**长度**： $|abc| = 3$

字符串的**连接**： $(ab) * (cd) = abcd$

字符串的**反转**： $(abc)^R = cba$

空词： ϵ , 长度为0

语言：给定字母表上一些字符串的集合

DFA 有限自动机

$$(Q, \Sigma, \delta, s, F)$$

- Q 是状态集，有限
- Σ 是字母表，有限
- δ 是转移函数： $Q * \Sigma \rightarrow Q$
- $s \in Q$ 是起始状态

- $F \subseteq Q$ 是接受状态集

输出：输入读完处于接受状态就接受，否则拒绝

对于一个有限自动机M，它接受的所有字符串构成一个**语言A**，也叫M**识别A**

如果存在DFA识别语言A，则A是**正则语言**，一个DFA只有一个语言，但是一个语言可以被多个DFA识别

DFA的设计

正则语言

设A,B都是 Σ 上的正则语言，那么 $A \cup B, A \cap B$ 都是正则语言

证明 $A \cup B$:构造一个新的DFA，同时识别A的DFA和B的DFA， $F = F1 \times Q2 \cup Q1 \times F2$ ，只要M1 接受，或者 M2 接受，新机器就接受。

证明 $A \cap B$:构造 $F = F1 \times F2$ ，M1 接受，并且 M2 也接受，新机器才接受。

正则运算

- 并: $A \cup B = \{x | x \in A \text{ 或 } x \in B\}$
- 连接: $A \circ B = \{xy | x \in A \text{ 且 } y \in B\}$
- 星号: $A^* = \{x_1 x_2 \dots x_k | k \geq 0 \text{ 且 每个 } x_i \in A\}$

三种运算均为封闭

非确定型有限自动机 NFA

每步可以有0-多种方式进入下一步，也可以是 ϵ 表示不读任何输入就转移过去

计算方式：只要存在一条接受路径，NFA就接受

形式定义：仍是一个五元组， $(Q, \Sigma, \delta, s, F)$ ，和DFA很像，区别在于转移函数： $\delta: Q * \Sigma_{\epsilon} \rightarrow P(Q)$ ，输出是一个集合

NFA 和 DFA 等价

每个 NFA 都有一台等价的 DFA。

NFA 只是让设计更方便，但表达能力没有变强。

NFA的确定化：子集法：

1. 先把NFA的**起始状态**和其他经过**任意空输入能到达的状态**的集合作为DFA的**起始状态**
2. 然后在已有的状态输入01（包括空输入），如果产生新集合就记作一个新状态
3. 直到没有新集合产生
4. 这时观察已有集合，如果集合中包含NFA的接受状态，那么这个集合就是DFA的接受状态

正则表达式

R是：

$$\begin{aligned}
 & a, a \in \sum \\
 & \varepsilon \\
 & \emptyset \\
 & (R_1 \cup R_2) \\
 & (R_1 \circ R_2) \\
 & (R_1^*) \\
 & R_1 \text{ 和 } R_2 \text{ 都是正则表达式}
 \end{aligned}$$

则R是正则表达式。

语言 A 正则 $\Leftrightarrow A$ 可以用正则表达式描述

语言 A 正则 $\Leftarrow A$ 可以用正则表达式描述 证明很简单

语言 A 正则 $\Rightarrow A$ 可以用正则表达式描述，构造GNFA

GNFA 广义非确定有限自动机

和NFA区别是，转义箭头可以用任何正则表达式

证明需注意：起始状态没有射入箭头；唯一接受状态无射出箭头，一个一个去掉中间状态

非正则语言：泵引理证明

如果存在数 p ，对任意一个长度不小于 p 的 w ，使得无论怎么分割 $w=xyz$ ，只要满足 $|y|>0$ 和 $|xy|\leq p$ ，任意的 i 让 $xy^i z$ 属于语言，那么这个语言就是正则语言

证明方法：用反证法，先假设是正则，然后选择一个 i 让 $xy^i z$ 不属于语言，从而得证

ch3 图灵机 TM

图灵机定义

$$\begin{aligned}
 & (Q, \sum, \Gamma, \delta, q_0, q_a, q_r) \\
 & Q \text{ 是状态集} \\
 & \sum \text{ 是输入字母表，不含空白符} \\
 & \Gamma \text{ 是带字母表，} \sum \text{ 是它的子集，包含空白符} \\
 & \delta \text{ 是转移函数} \\
 & q_0 \text{ 是起始状态，} q_a \text{ 是接受状态，} q_r \text{ 是拒绝状态，} q_a \neq q_r
 \end{aligned}$$

初始化

输入带：把输入串 w 放在最左段 n 格，后面的部分补充空格

读写头指向最左端

运行

根据转移函数运行，比如 $\delta(q_0, 0) = (p, \#, R)$ 表示处于状态 q_0 且读到符号 0 时，会把格子改成符号 $\#$ ，然后想方向 R (右) 移动，进入下一个状态 p

如果在最左端左移，那么读写头保持不动

判定器和语言分类

如果TM进入接受或拒绝状态，则停机；否则会一直运行

如果M对所有输入都停机，那么M是**判定器**

可判定语言(递归语言)：某个判定器的语言，对属于语言的输入接受，不属于的拒绝

可识别语言(递归可枚举语言)：属于语言的输入接受，不属于的可能拒绝也**可能不停机**

格局

对图灵机 $M = (Q, \Sigma, \Gamma, \delta, q_0, q_a, q_r)$, $q \in Q, u, v \in \Gamma^*$, 那么格局 uqv 表示：

1. 当前状态为q
2. 存储带上的字符串是uv
3. u是读写头左边的字符串，v是从读写头开始向右的字符串

图灵机举例

图灵机的描述

形式水平的描述（状态图或转移函数）

实现水平的描述（读写头的移动和改写）

高水平描述（日常语言）

用带引号的文字段表示图灵机

输入总是字符串，有时要输入数，图等，要讲对象编码成字符串

图灵机的变形

非确定型图灵机 NTM：每一步有多个分支，如果对所有输入和所有分支都停机，则为判定NTM

如果有其中一个分支接受了w，那么NTM接受w

ch4 可判定性

凡是直觉上可以用算法完成的事情，都可以由图灵机完成

可判定：符合输入的回答对，不符合的回答错，都能对输入给出一个明确的答案，不会卡死循环

停机问题：不存在一个万能程序，可以判断所有程序在所有输入上会不会停机

DFA

比如

$$A_{DFA} = \{ \langle B, w \rangle \mid B \text{ 是 DFA, } B \text{ 接受 } w \}$$

我们可以写出一个算法：检查B是否为DFA，把w输入给B，然后看停在哪个状态，如果接受，算法输出Yes，否则输出No，所以这个是可判定的

NFA

$$A_{NFA} = \{ \langle B, w \rangle \mid B \text{ 是 NFA, } B \text{ 接受 } w \}$$

因为NFA都能转换成一个等价的DFA，所以这个也是可判定的

空性质

$$E_{DFA} = \{ \langle A \rangle \mid A \text{ 是 DFA, } L(A) = \emptyset \}$$

A的语言不是空集当且仅当存在从起始状态到接受状态的某个路径

算法设计：标记起始状态，重复：对**已标记状态的相邻未标记状态**做标记，直到没有新标记，最后如果标记状态中没有接受状态，则算法接受，否则拒绝。**因此它是可判定的。**(图搜索算法)

TM成员测试

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ 是一个 TM, 且接受 } w \}$$

不可判定，但是是图灵可识别的

证明图灵可识别：

构造识别器U，输入 $\langle M, w \rangle$ ，那么U和M的状态相同，接受、拒绝或者不停机。

U的语言就是 A_{TM} ，所以 A_{TM} 是图灵可识别

证明不可判定：

先假设可判定，则存在一个判定器H，对M接受w，H接受，M不接受w，H拒绝

构造图灵机D，运行 $H(\langle M, \langle M \rangle \rangle)$ ，如果H说M接受自己的编码 $\langle M \rangle$ ，那么D拒绝，反之接受。

总之，如果M接受自己，D就拒绝，M拒绝D就接受

现在看D是否接受 $\langle D \rangle$ 自己：D接受自己的编码当且仅当H拒绝 $\langle D, \langle D \rangle \rangle$ ，也就是D拒绝自己，矛盾。所以不存在这样的H。

直觉：**如果存在一个万能算法能判断任意机器是否接受任意输入，那我就造一个机器专门和它预测的结果相反，最后让这个机器预测自己，就爆炸了。**

A_{TM} 的补不是图灵可识别的

先证：如果A和A的补都是可识别的，那么A是可判定的。

假设T识别A，Q识别A的补，构造判定器R，同步模拟T和Q，如果T接受则接受，Q接受则拒绝，一定会停机，所以A是可判定的。

结合上面的结论，**如果A的补是可识别的，又因为A是可识别的，所以A可判定，但是A已经证明不是可判定的，所以A的补一定不是可识别的。**

语言的关系

$$\text{正则语言} \subseteq \text{上下文无关语言} \subseteq \text{可判定语言} \subseteq \text{图灵可识别语言} \subseteq P(\sum^*)$$

一元多项式整数根问题：可判定，多元多项式整数根问题：不可判定

ch7 计算复杂性

时间复杂度

$f(n)$ 是M在所有长为n的输入上运行的最大步数

如果 $f(n)$ 是M的运行时间则称M在时间 $f(n)$ 内运行，或M是 $f(n)$ 时间图灵机

$$f(n) = O(g(n))$$

n 足够大时， $f(n)$ 增长速度最多和 $g(n)$ 一样快

$$f(n) = o(g(n))$$

$f(n)$ 的增长速度严格比 $g(n)$ 慢，最后相比 $g(n)$ 可以忽略

时间复杂性类：

$$TIME(t(n)) = \{L \mid \text{存在 } O(t(n)) \text{ 时间 } TM \text{ 判定 } L\}$$

所有能在 $O(t(n))$ 时间内被图灵机判定的语言，组成一个集合

定理：时间 $o(n \log n)$ 的单带图灵机判定的语言是正则语言

$$TIME(o(n \log n)) \subseteq \text{正则语言类} \subseteq TIME(n) \subseteq TIME(o(n \log n))$$
$$\text{正则语言类} = TIME(n) = TIME(o(n \log n))$$

不同模型运行时间比较

单带与多带运行时间

现在对 $\{0^k 1^k \mid k \geq 0\}$ 进行讨论，它有 $O(n)$ 时间的双带图灵机，具体实现略

定理：设函数 $t(n) \geq n$ ，则每个 $t(n)$ 时间多带TM和某个 $O(t^2(n))$ 时间单带TM等价

非确定判定器

非确定判定器N的运行时间 $f(n)$ 是**所有**长为 n 的输入中，**所有**分支的最大步数。

NTM的运行时间：**设 $t(n) \geq n$ ，则每个 $t(n)$ 时间NTM都有一个 $2^{O(t(n))}$ 单带确定TM与之等价。**

$$NTIME(t(n)) \subseteq TIME(2^{O(t(n))})$$

P与NP

P类

P是**单带确定TM在多项式时间内可判定**的问题， $P = \cup_k TIME(n^k)$

P类对于所有单带确定TM等价的模型，P不变

大致对应于计算机上实际可解的问题

NP类

NP是**单带非确定TM在多项式时间内可判定**的问题， $NP = \cup_k NTIME(n^k)$

$$EXP = \cup_k TIME(2^{O(n^k)})$$
$$P \subseteq NP \subseteq EXP$$
$$P \subsetneq EXP$$

实际问题

属于P类：GCD，模p指数运算，上下文无关语言，素性测试

增加空间复杂性来减少时间复杂性

NP类可以快速验证，别人给答案去验证正确性

NP完全性及NP完全问题

NP完全问题：若这些NP问题中任意一个找到P时间算法，则 $P=NP$ ，所有NP问题都能规约到它

合取范式

布尔变量：取值为1,0

布尔运算：AND,OR,NOT

布尔公式： $\phi = x \wedge y$, 如果存在赋值使 $\phi = 1$ 则 ϕ 可满足

文字：变量或变量的非

子句：用 $\vee$ 连接的若干文字

合取范式(cnf)： $\wedge$ 连接的若干子句

k-cnf: 每个子句的文字数不大于k

可满足问题SAT

可满足性问题： $SAT = \{ \langle \phi \rangle \mid \phi \text{ 是可满足的布尔公式} \}$ NP完全

二元可满足性问题： $2SAT = \{ \langle \phi \rangle \mid \phi \text{ 是可满足的 } 2cnf \}$ $\in P$

三元可满足性问题： $3SAT = \{ \langle \phi \rangle \mid \phi \text{ 是可满足的 } 3cnf \}$ NP完全

SAT的NP完全的意思是，可以**快速验证**赋值是否让公式为真，**任何NP问题**都可以在多项式时间内**转化成一个SAT**。

证明二元可满足问题 $\in P$ ：当子句中含有单文字x时，直接令x为1，然后可以删掉含x的子句和子句中的非x；没有单文字时，任选一个文字赋值0,1各一次，如果都失败则不可满足，否则可满足

目前不知道3SAT是否属于P

归约引理

定义多项式时间可计算函数 $f: \Sigma^* \rightarrow \Sigma^*$, 输入字符串w, 输出f(w)

如果存在一个f, 使

$$\forall w \in \Sigma^*, w \in A \Leftrightarrow f(w) \in B$$

那么A可多项式时间映射规约到B($A \leq_P B$)，就是用B的算法解决A

C-L定理：对任意的 $A \in NP$ 都有 $A \leq_P SAT$ ，就是所有的NP问题都能转化成SAT

规约引理：若 $A \leq_P B, B \in P$, 则 $A \in P$

由引理得，如果 $SAT \in P$, 那么 $NP = P$

设B有P时间判定器M，流程为：输入w，经过f映射为f(w),然后将f(w)代入M得到结果。如果f是 n^a 时间归约，M是 n^b 时间判定器，那么 **$N=$ 输入w，计算M(f(w))，输出运行结果的时间计算：**

$$T_N(n) = O(n^a) + O(n^{ab})$$

CLIQUE

$CLIQUE = \{ \langle G, k \rangle \mid G \text{ 是有 } k \text{ 团的无向图} \}$, 表示图中有k个顶点两两都有边相连

定理: $3SAT \leq_P CLIQUE$

证明: 设 ϕ 有k个子句, 令 $f(\phi) = \langle G, k \rangle$, G有k组节点, 每组3个, 同组节点无边, 相反标记无边相连。现在要证: $\langle \phi \rangle \in 3SAT \Leftrightarrow \langle G, k \rangle \in CLIQUE$

从 $\langle \phi \rangle \in 3SAT$ 出发:

$$\begin{aligned} & \langle \phi \rangle \in 3SAT \\ \Leftrightarrow & \text{存在一组赋值使 } \phi = 1 \\ \Leftrightarrow & \text{一定存在 } k \text{ 团, 因为每组都能找到真顶点得到 } k \text{ 团} \\ \Leftrightarrow & f(\phi)(\langle G, 3 \rangle) \in CLIQUE \end{aligned}$$

ϕ 的长度是 $O(n)$, 则G的边数是 $O(k^2)$, 可见构造可在k的多项式时间内完成

NP完全性

B称为**NP完全的**(NPC), 满足: $B \in NP, \forall A \in NP, \text{ 都有 } A \leq_P B$

定理1: $A \leq_P B, B \in P \Rightarrow A \in P$

定理2: 如果B是NPC, $B \in P$ 则 $P = NP$

定理3: 如果B是NPC, $B \leq_P C$, 且 $C \in NP$, 那么C是NPC

C-L定理证明补充

C-L定理: SAT是NP完全问题

证明步骤: $SAT \in NP$, 已经证明可以快速验证, 现在要证 $\forall A \in NP, A \leq_P SAT$

把一台NTM对输入w的接受格局序列编码成一个布尔公式 ϕ_w

NTM 接受 $w \Leftrightarrow$ 能填好 N 在 w 上的表 $\Leftrightarrow$ 公式 ϕ_w 是可满足的

$$\phi = \phi_{cell} \wedge \phi_{start} \wedge \phi_{move} \wedge \phi_{accept}$$

ϕ_{cell} : 限制每格有且只有一个字符

ϕ_{start} : 第一行是合法起始状态

ϕ_{move} : 每行是上一行的合法结果

ϕ_{accept} : 存在接受状态

$$\forall w, w \in A \Leftrightarrow \langle \phi \rangle \in SAT$$

可以得到 $A \leq_m SAT$

$\leq_m$ 表示映射归约, 指可以把 A 的每个实例 w 映射为 SAT 的实例 $\langle \phi \rangle$

如果公式长度是输入长度的多项式: $|\langle \phi \rangle| \leq |w|^c \text{ (} |w| = n, |\phi| = O(n^{2k}) \text{)}$

那么有 $A \leq_P SAT$

用变量 $x_{i,j,s}$ 表示 ϕ , 这个变量表示i行j列是否填了符号s, 构造 $\phi_{cell}, \phi_{start}, \phi_{accept}$

ϕ_{move} 确定每行是上一行的合法结果, 只需判断每个2*3窗口是否合法, 2*2不能正确判断

3SAT是NP完全的

$$\phi = \phi_{cell} \wedge \phi_{start} \wedge \phi_{move} \wedge \phi_{accept}$$

从这个公式出发, 把它改造成3CNF

可以通过引入新变量把k文字子句最后变成k-2个3文字子句

ϕ_{cell} 的公式处理较为特殊，要用到分配律。设合法窗口有M个，原长度为 $6Mn^{2k}$ ，分配律后变为 $M6^Mn^{2k}$ ，再变成3cnf，长度为 $3(M-2)6^Mn^{2k}$ ，从而证明3SAT是NP完全的

HP哈密顿路径

$$3SAT \leq_P HP$$

UHP无向哈密顿

$$HP \leq_P UHP$$

把哈密顿图中的每一个点换成三个点 v_{in}, v_{mid}, v_{out}

起点只变成out一个,终点只变成一个in

UHC无向哈密顿圈

$$UHP \leq_P UHC$$

在无向哈密顿图的前提下，添加一个新点a，连接起点s和终点t，这样就变成了UHC

TSP旅行售货员问题

构造一个对应原来图G的完全图G'，定义边的权重：在原来UHC中的边权重为1，不在的为2，所以总权重的回路就是原来的UHC

KS问题

给一个集合，里面有一些数字，能否选出一个子集，元素和等于目标t

证明： $3SAT \leq_P KS$